

Os skill 20

20(a) Capture SIGTSTP and Suspend Processes

What it does: Captures SIGTSTP, suspends a foreground process, updates its state, and resumes it later using SIGCONT.

```
sri_dhruthi@Dhruthi: ~/Skill x + v
GNU nano 8.7.1 skill20a.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

pid_t child_pid;

void handle_sigtstp(int sig)
{
    printf("\nSIGTSTP received.\n");
    if (child_pid > 0)
    {
        kill(child_pid, SIGSTOP);
        printf("Foreground job suspended.\n");
    }
}

int main()
{
    int status;

    child_pid = fork();

    if (child_pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (child_pid == 0)
    {
        signal(SIGTSTP, SIG_DFL);

        printf("Foreground job started. PID = %d\n", getpid());
        printf("Job is running...\n");
    }
}
```

```
sri_dhruthi@Dhruthi:~/Skill$ nano skill20a.c
sri_dhruthi@Dhruthi:~/Skill$ gcc skill20a.c -o skill20a
sri_dhruthi@Dhruthi:~/Skill$ ./skill20a
Shell monitoring job PID = 639
Foreground job started. PID = 639
Job is running...

SIGTSTP received.
Foreground job suspended.
Job state: STOPPED
Sending SIGCONT...
Job completed.
Job state: COMPLETED
sri_dhruthi@Dhruthi:~/Skill$ |
```

20(b) Process Groups and Terminal Control

What it does: Creates a process group, assigns a group ID, coordinates signals using the process group, and restores control to the parent

```
sri_dhruthi@Dhruthi: ~/Skill x + v
GNU nano 8.7.1 skill20b.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        /* Create a new process group */
        setpgid(0, 0);

        printf("Child process created.\n");
        printf("PID = %d\n", getpid());
        printf("PGID = %d\n", getpgrp());

        sleep(3);

        printf("Child process completed.\n");

        return 0;
    }

    /* Assign child to its own process group */
    setpgid(pid, pid);

    printf("Parent/Shell process.\n");
}
```

```
JOB State: COMPLETED
sri_dhruthi@Dhruthi:~/Skill$ nano skill20b.c
sri_dhruthi@Dhruthi:~/Skill$ gcc skill20b.c -o skill20b
sri_dhruthi@Dhruthi:~/Skill$ ./skill20b
Parent/Shell process.
Child PID = 684
Child PGID = 684
Sending signal to process group...
Child process created.
PID = 684
PGID = 684
Child process completed.
Child finished.
Terminal control restored to shell.
```

